



DEGREE PROJECT, IN COMPUTER SCIENCE , FIRST LEVEL
STOCKHOLM, SWEDEN 2015

A benchmark of algorithms for the Professor's Cube

MATTIAS DANIELSSON

KTH ROYAL INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTER SCIENCE AND COMMUNICATION



A benchmark of algorithms for the Professor's Cube

MATTIAS DANIELSSON

Degree Project in Computer Science, DD143X

Supervisor: Michael Schliephake

Examiner: Örjan Ekeberg

Abstract

Rubik's Cube is a well known puzzle that has entertained for decades. This thesis studies two relatively new phenomenons; the 5^3 cube, also known as the Professor's Cube , and the practice of speedcubing. This report presents two existing algorithms developed by speedcubers that enable human executors to quickly solve the puzzle. The algorithms are implemented and their results compared in order to show which one requires fewer steps to solve the puzzle. The conclusion is that it is faster to use the original Davenport algorithm instead of the newer version by Monroe if the number of twists is your limiting factor.

Contents

Contents	4
1 Introduction	1
1.1 Problem Statement	1
2 Background	3
2.1 The Cube	3
2.2 Definitions	4
2.3 Mathematical Properties of The Cube	7
2.4 Solution Algorithms	7
3 Method	11
3.1 Implementation	11
3.2 Measurements	12
4 Results	13
5 Discussion	15
5.1 Analysis	15
5.2 Sources of Error	16
5.3 Conclusion	16
5.4 Areas of Future Study	16
Bibliography	19

Chapter 1

Introduction

The Rubik's Cube is a well-known puzzle that came to be the iconic toy of a generation. Since the release of the original cube in the eighties, several new variations have been brought to the market [1]. Additionally, the last years have seen the upbringing of competitions in speed-solving the cubes [2]. These two processes have together created a set of methods and algorithms to improve the contestants' chances of winning. The algorithms are not always created with the aid of relevant science, but are instead likely to be the product of less formal methods. This makes the algorithms interesting in several aspects, and this thesis is focused on evaluating the performance properties of some of these algorithms.

1.1 Problem Statement

The main purpose of this study is to compare two algorithms used by human executors to solve the Professor's Cube. As the algorithms are used by the contestants who continually twist the cube while running the algorithm, there is value in knowing which algorithm requires the least amount of steps to reach the end state.

In this thesis, I have chosen to study two versions of the popular solution algorithm constructed by Jacob Davenport [3]. The first version is the original Davenport's algorithm, and the second version is the modified version contributed by Matthew Monroe. This investigates whether or not the modified version is more or less efficient than the original.

The study has also been limited to consider each twist as taking constant time. It would be interesting to build a more complicated model where consideration was taken of whether or not the executor would be required to change the grip of the cube. This has however not been considered in this study.

Chapter 2

Background

In this section the background for the problem statement will be presented and explained. I will discuss what the cube is, what speedcubing is and why there is an interest in exploring the problem statement.

This paper tangentially touches on the research by Echeverra and Westermarck in their 2013 thesis[4]. Echeverra and Westermarck focused on two algorithms, the reduction method and the "Big Cube" method, for speedsolving Rubik's Revenge. Rubik's Revenge is the trade name given to the 4x4x4 Rubik's style cube. As stated above, this thesis will focus on the larger 5x5x5 cube also known as the Professor's Cube . Also, this study is focused on finding differences in two versions of the same algorithm, in contrast with the earlier study which considered two wildly different solution strategies.

2.1 The Cube

Rubik's Cube was originally designed by the Hungarian architecture professor Ernő Rubik in 1974 [1]. The cube was not originally intended as a toy or a puzzle; Rubik simply wanted a model able to demonstrate three dimensional geometry. Released to the greater public in 1980, it was immediately named Toy of the Year, and has to date sold in over 350 million (official) copies [1].

Multiple variations to the original cube were quickly developed, including cubes with additional corner pieces and "cubes" with other geometrical shapes, such as the pyramid-formed toy sold under the name Pyraminx [5]. This thesis is going to focus on the cube that was patented in 1986 by Udo Krell [6], and is now sold under the trademark "the Professor's Cube ". It is a very simple variation on the original puzzle; the Professor's Cube is merely a 5x5x5 version of the original 3x3x3 cube created by Rubik.

Speedcubing

”Speedcubing” refers to the activity of trying to solve cubes in the absolute least amount of time. The phenomenon of speedcubing was created in 1999 when a few people with a common interest in solving the cube quickly met in a Rubik’s Cube game online [2]. Some of the people from that group would eventually found the World Cube Association in 2004, creating the first organisation dedicated to arranging and regulating speedcubing competitions [2]. Today, the organisation arranges competitions in over 50 countries. According to WCA, the current world speed record solving the Rubik’s Cube is 5.5 seconds, and the record for the Professor’s Cube is 48.42 seconds [7].

The increasing competitiveness of speedcubing has caused the community to develop several algorithms that help speedcubers to reduce the time to find the solution. These algorithms were often developed using experience, intuition and trial-and-error, with varying levels of influence from scientific analysis and methods. Two of these algorithms has been chosen for study, and will be further described in section 2.4.

Scrambling

Official contests carried out by the WCA or its partners are required to use cubes created from scrambles generated by the official scramble generator available from WCAs website [8].

2.2 Definitions

For the purpose of the discussion in this thesis, let n -cube denote a Rubik’s style cube with $n \times n \times n$ *cubies*. Each cubie has an unique color on each visible face. A set of cubies that can move together in a single twist is called a *slice*. A cubie is called an center cubie if it is at the center of a *face* and has one visible color. A cubie is called an edge cubie if it has two colors and is in the same slice as a center cubie.

The previous nomenclature is enough to describe the classical Rubik’s Cube, but in order to describe the Professor’s Cube I need a few additional terms. An *wing* cubie has two colors but does not exist in the same slice as a center piece. The four single colored cubies horizontally and vertically between the center piece and the four edge pieces are called *cross* cubies. The remaining four cubies diagonally between the center piece and the corner cubies are called *point* cubies.

In the Rubik’s Cube community, there are two main metrics for determining step length; full- turn metric and quarter-turn metric [9]. Quarter turn

counts the number of quarter turns required to perform an algorithm. Full turn considers a turn to be a single turn independently of whether the move required a 90° turn or a 180° turn. For the purpose of this thesis the full-turn metric will be used.

A twist is the twist of a single slice in the cube. A rotation is a rotation of the entire cube.

Cube Faces

The cube, like most cubes, has six faces. It is often practical to use a relative system to denote each one, and the one used in this thesis can be described as follows. It is quite straight-forward.

Observe the cube orthogonally on one of its sides. The face toward the observer is the *Front* face, *a*. The face upwards is the *Up* face and the face down is the *Down* face. The face to the right is the *Right* face and the left face is the *Left* face. Unsurprisingly, the bottom face is called the *Down* face.

Facelets

The faces of the individual faces of the cubies are called facelets. The facelets has names in order to support easier addressing of facelets on a cube face. See Figure 1 for the placement of the facelets. The facelets are named Corners, Wings, Edges, Points, Crosses, and Centers. The facelet names are presented in Figure 2.1 by their abbreviations.

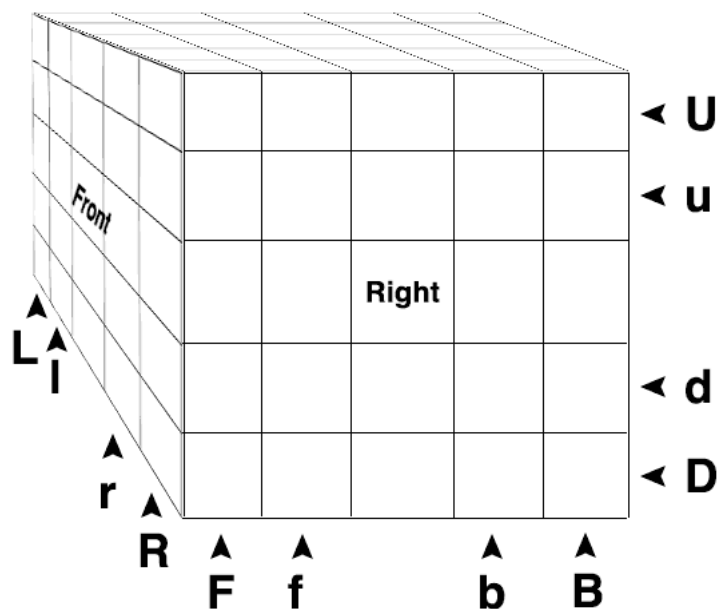
Co	W	E	W	Co
W	P	Cr	P	W
E	Cr	Ce	Cr	E
W	P	Cr	P	W
Co	W	E	W	Co

Figure 2.1: Facelet names

Turn notation

This thesis will use the turn notation adopted by the WCA[8].

There are twelve slices that can be turned on the Professor's Cube, and each slice can be turned clockwise, counter-clockwise and a half turn. This creates a total of 36 possible transformations available in each step.



U is the top slice.

u is the slice immediately below the top slice.

H is the horizontal middle slice.

D is the bottom slice.

d is the slice immediately above the bottom slice.

F is the front slice.

f is the slice immediately below the front slice.

B is the back slice.

b is the slice immediately in front of the back slice.

L is the leftmost slice.

l is the slice immediately to the right of the left slice.

R is the rightmost slice.

r is the slice immediately to the left of the right slice.

Using these letters in the context of a transformation indicates that the specified slice should be rotated one quarter of a turn clockwise considered straight onto the outermost parallel face.

The prime sign is added to the letter to indicate that the turn should be counter-clockwise.

A "2" is added after the letter to indicate that the turn should be a half-turn.

Example The transformation sequence (D' f r2) represents the sequence:

1. Turn the D-slice counter-clockwise one quarter turn.
2. Turn the f-slice clockwise one quarter turn.
3. Turn the r-slice either way one half turn.

2.3 Mathematical Properties of The Cube

The Rubik's Cube is a very good example of group theory. The possible configurations of the cube under the basic twist operation creates a permutation group. Each basic twist of the Rubik's Cube constitute a subgroup of the permutation group.

The issue of solving the cube optimally has been an interest of computer scientists since the inception of the first cube. In 2010 Rokicki et al. produced a definite proof that the Rubik's Cube could be solved with at most 20 steps from all legal permutations. This was proved computationally by optimally solving each possible permutations, a feat which required 35 years of CPU time to complete. [9] This number is often referred to as God's Number. The God's Number for the Professor's Cube has not been determined, and it is unlikely that such a feat will be accomplished within a reasonable timeframe, considering the significantly larger number of possible permutations of the Professor's Cube .

For reference, the number of possible permutations of Rubik's Cube is 4.3×10^{18} , or approximately 43 quintillion[10]. The Professor's Cube has a significantly higher number of permutations, counting in at 1.24×10^{92} [11]. The large number of permutations, of which only one is a solution, puts a stringent demand on efficiency in any solution algorithm.

2.4 Solution Algorithms

The notion of *God's Algorithm*, a concept closely related to God's Number, has been discussed within the speedcuber community. God's Algorithm is any

algorithm that is able to create solutions that uses the least possible number of steps to reach the end state. This would require knowledge of the optimal path from any given configuration, which is why the concept is attributed to an all-knowing being.

The first algorithm published that could be called God's Algorithm was published by Thistlethwaite[12] in 1981, only a year after the general release of the original Rubik's Cube . Thistlethwaite's algorithm did not however guarantee that the solution was the shortest, and so Richard Korf released a correct solution in 1985 [13] that uses iterative deepening to find a guaranteed solution. For the continued discourse no further details are required about these algorithms are needed.

One of the solution algorithms developed by the speedcubing community for the use of human executors to solve the Professor's Cube is known as Davenports algorithm[3].

Davenports algorithm is a reduction solution. That is, the algorithm is not able to fully solve the Professor's Cube on its own, instead it reduces the cube to a state where the computer can use their favourite 3^3 -cube solution to complete the process. This clearly shows the intent of the algorithm author for the algorithm to be executed by human executors.

Davenports Algorithm

An overview of Davenports algorithm will be given here. To get all the details, the reader is advised to read the original source[3]. This overview is not detailed enough to use it to implement the algorithm, but is intended for readers to get an understanding of how the algorithm is specified. The original text is quite dense, and any attempt to reproduce the description would turn into a gigantic quote and a cheaply inflated word count.

Phase 1 The six inner axes are rotated so that the crosses and points of the top face is the same color as the top center. The method to accomplish this is not specified by Davenport.

Phase 2 The bottom cross(the cross and center cubies) is moved into position by rotating the inner horizontal axes until the desired facelets are placed on the front point cubies. The moves $(R' r' D^2 R r)$ and $(R' r' D R r)$ are then used to place the facelets at the bottom.

Phase 3 For each of the faces (Front Left Back Right), the slice d and F are rotated until the upper points are of the correct color.

Phase 4 The top and bottom crosses are completed by rotating H and R until the Cr3 position on face R are the top or bottom color. The precomputed sequence (H' F' f' H F f) is used to move the facelet up and the sequence (F f H' F' f' H' F f H F' f' H) moves the facelet down.

Phase 5 Search for a face that is not up or down that has like-colored facelets on the correct face, and then orient them correctly using two sequences. Repeat until all crosses are fixed.

Phase 6 The wings are associated with their corresponding edge pieces, again through the sequential application of a series of precomputed sequences.

(Phase 7) The cube has now been reduced, and any 3^3 cube solver can be used to solve the cube fully.

Monroes Modification

Monroes version modifies step five by providing a series of precomputed transformations:[14]

Phase 5 The crosses on the Front, Left, Back, and Right faces are completed by applying one of the precomputed sequences: (F2 f2 H F2 f2 H'), (H L2 l2 H' L l H' L l H' L2 l2 H L l H L l) and (U' u' R' H L2 l2 H' L l H' L l H' L2 l2 H L l H L l R U u).

Chapter 3

Method

This section describes how the algorithms were implemented and which considerations and applications were taken to accommodate the somewhat informally defined algorithms for a machine.

3.1 Implementation

Cube Model

The cube is implemented in the Java programming language. Java was chosen for its maturity and reasonable speed. The cube is represented as an array of six 5x5 matrices that holds the color for each facelet on the cube. A set of transformation functions ensures that only valid twists can be exerted onto the cube. The cube can only be modified by applying sequences of transform atoms (F, f' etc) onto the representation.

Algorithm implementation

The algorithms were implemented according to the specification. In some cases extensive searches used added to find the next facelet on certain orbits. This is required when the instruction seems to request that the human executor's intuition is used to find suitable moves.

The algorithm is built on top of the model of the cube, in order to ensure that it only modifies the cube representation through the use of transformation sequences.

The implementation reduces the output to step 6, then stops. The 3^3 is not attempted.

Handling long-running solvers

A timeout was added so that when the running time for a solver exceeded 2 seconds it was killed and the attempt marked as a failure. The limit of 2 seconds was chosen after a few test runs showed that increasing the limit further did not increase the number of solved cubes.

Note on the first step

The first step of the algorithms are to make sure that eight of the nine of the center facelets on the UP face are of equal colors. Davenport writes that this step is so simple that no specification of the procedure used to reach this state is necessary.[3] The implementation simply brute-forces this step. Moves are not counted during the brute-forcing.

Verification

A solution is considered successful when facelets not belonging to the 3^3 problem has a single color per face. Additionally, the number of facelets of each color is counted for verification, and adjoining edge cubies are checked for consistency.

3.2 Measurements

The cube representation counts the transformation atoms passed to the cube's transformation code. There is no codepath that updates the internal representation that does not pass by the counter, except during initialization.

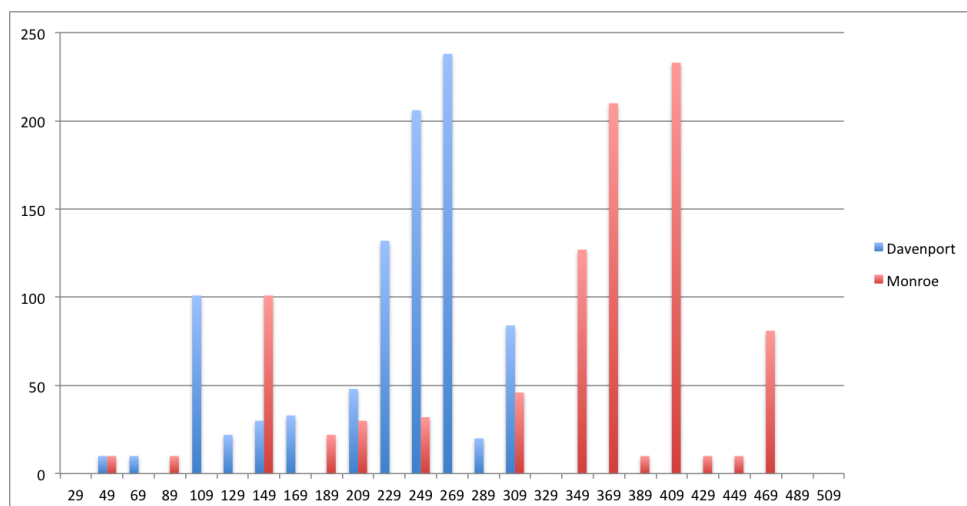
Both models were given 1000 scrambles from the official WCA scrambler and the results were collected and analyzed.

Chapter 4

Results

	Davenport	Monroe
Failures	68	70
Average Steps	218.6	331.4
Median Steps	232	352
Standard Deviation	62.06	99.44

Figure 4.1: The results of a run on 1000 official scrambles



	Davenports	Monroes
0-29	0	0
30-49	9	10
50-69	10	0
70-89	0	10
90-109	101	0
110-129	22	0
130-149	30	101
150-169	33	0
170-189	0	22
190-209	48	30
210-229	132	0
230-249	206	32
250-269	238	0
270-289	20	0
290-309	84	46
310-329	0	0
330-349	0	127
350-369	0	210
370-389	0	10
390-409	0	233
410-429	0	10
430-449	0	10
450-469	0	81
470-489	0	0
490-509	0	0

Figure 4.2: Frequency table of a run of 1000 scrambles

Chapter 5

Discussion

In this section the results are analysed and at the end some conclusions will be drawn. The results are first examined naively before being discussed in a more critical manner. Limitations and sources of error are highlighted, and finally a conclusion is drawn from the presented factors.

5.1 Analysis

The results indicate with certain clarity that the original Davenport implementation is the more efficient, requiring on average one third less moves in order to reach a reduction. Davenport's was also more consistent in the number of steps required to complete, as indicated by the lower deviation.

Algorithm Implementation

It proved to be a challenging task to implement the algorithm in a meaningful way. The main concern was the fact that some parts of the algorithm essentially did not offer any guidance with how to transfer certain facelets to certain positions, with the explanation that it would be simple for an observer. The issue is not that the operation is hard to implement(it has been done several times), it is that the objective for this thesis is to implement an algorithm intended for humans. With that in mind it is hard to apply any of the existing algorithms, and instead one must translate the operation into something more intuitive. A common decision was to just try every combination of some moves until the facelet arrived at the correct destination.

These attempts caused the solver to not terminate in some instances within the 2 second timeframe. The results indicate that 30 permutations out of 1000 ended up exceeding the time limit, so this was not a too common problem. It is

however not known whether the permutations timed out because of inefficiency or because of defects in the algorithm or in the implementation.

5.2 Sources of Error

The simple expensive searches used in some parts of the algorithm are perhaps way too simple. There are most likely more efficient ways of finding facelets that does not waste as many twists.

The verification model in the implementation does not guarantee that the cubes integrity is conserved over all transformations: it is still possible that the solver is able to translate over illegal positions.

This solution does not attempt to solve the reduced cube. It might feel unintuitive, but it is possible that the two solution algorithms produces reductions of different difficulty. If so, the results would be misleading.

5.3 Conclusion

In the objective I asked if there would be a difference in efficiency between the algorithms. The data clearly indicates a positive; Davenport's algorithm has a clear advantage in terms of steps required to execute it.

However, one must not rule out the more simple algorithm by Monroe; by using Monroe a human executor can learn to execute the algorithm with far less steps than what Davenport's would require. Further research is required to determine how the algorithms perform when a more complex measurement method is used, which would take account for time required to reorient the cube and more.

In conclusion, I recommend that serious speedcubers focus on the Davenport algorithm in order to increase efficiency and reduce solution time, conditional whether the number of twists required during the execution is the prime limiting factor for the executor.

5.4 Areas of Future Study

It would be interesting to continue to implement the solver for the reduction. This would hopefully yield an answer to whether or not the algorithms are able to generate reductions of varying complexity.

Additionally, a future study could implement a more realistic model for measuring the number of moves. It would be interesting to account for the fact that rotations are much more time-consuming than twists, and that twists

that are on the same axis as the previous twist are faster to execute by a person.

Bibliography

- [1] Rubiks Brand Ltd, *The history of the rubiks cube*. [Online]. Available: <https://eu.rubiks.com/about/the-history-of-the-rubiks-cube/> (visited on 04/14/2015).
- [2] World Cube Association, *About the WCA*. [Online]. Available: <https://www.worldcubeassociation.org/about>.
- [3] J. Davenport, *How to solve the 5x5x5 rubik's cube*, 2009. [Online]. Available: http://www.playagaingames.com/entertainment/5x5x5_solution/ (visited on 04/22/2015).
- [4] P. B. Echeverria and J. Westermarck, “Benchmarking rubik’s revenge algorithms”, Bachelor Thesis, KTH The Royal Institute of Technology, School of Communication and Computer Science, 2013.
- [5] I. Abu-Shumays and M. Abu-Shumays, *Polyhedral and sperical cubic puzzles*, US Patent 4,593,907, 1986.
- [6] U. Krell, *Three dimensional puzzle*, US Patent 4,600,199, 1986.
- [7] World Cube Association, *Results (WCA)*. [Online]. Available: <https://www.worldcubeassociation.org/results/regions.php>.
- [8] —, *WCA regulations*. [Online]. Available: <https://www.worldcubeassociation.org/regulations/>.
- [9] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, “The diameter of the rubik’s cube group is twenty”, *SIAM Review*, vol. 56, no. 4, pp. 645–670, 2014.
- [10] C. Bandelow, “The mathematical model”, in *Inside Rubik’s Cube and Beyond*, Springer, 1982, pp. 26–55.
- [11] D. Singmaster, “The 5^3 ”, *Cubic Circular*, no. 3, p. 18, 1982.
- [12] M. Thistlethwaite, *The 45-52 move strategy*, London CL VIII, 1981.
- [13] R. E. Korf, “Depth-first iterative-deepening: an optimal admissible tree search”, *Artificial intelligence*, vol. 27, no. 1, pp. 97–109, 1985.

- [14] M. Monroe, *Directions for solving the 5x5x5 (professor) cube*, 2014. [Online]. Available: <http://www.alchemistmatt.com/cube/5by5cube.html> (visited on 04/22/2015).

